Topic 4: API Authentication and Security

Basic Auth, Bearer Tokens, OAuth 2.0 & JWT Architecture

1. Document Control

Field	Specification Value
Document Title	Topic 4: API Authentication and Security Specification
Course Module	API Integration Hands-On for n8n (Level 3 - Enterprise Series)
Version	1.0 (Production Master)
Status	**APPROVED & VERIFIED**
Last Updated	2026-11-09
Prepared By	Lead Solutions Architect & Automation Engineering Team
Target Audience	Integration Engineers, API Developers, Security Engineers, Automation Trainees

2. Learning Concepts & Architectural Principles

In production enterprise architectures, unauthenticated endpoints do not exist. APIs safeguard proprietary resources and customer PII across multiple authentication topologies:

- **HTTP Basic Authentication:** Legacy standard encoding credentials as `Authorization: Basic base64(username:password)`. Suitable for simple microservices behind TLS.
- **Bearer Token Authentication:** Modern token-based authorization passing bearer access tokens in headers (`Authorization: Bearer <token>`).
- **OAuth 2.0 Delegation Protocol:** Industry standard enabling third-party applications to obtain limited access to an HTTP service on behalf of a resource owner without exposing user credentials.
- **JSON Web Tokens (JWT):** Stateless, digitally signed claims containers enabling cross-service identity propagation without repeated database lookups.

n8n Decoupled Credential Store

A core principle of security hygiene is separating code/canvas logic from cryptographic credentials:

- **Zero Plain-Text Leakage:** Exporting workflow JSON files completely strips sensitive passwords, API keys, and client secrets.
- **Environment Reusability:** A single Basic Auth or OAuth credential can power dozens of distinct workflows across an enterprise workspace.
- **Automated Lifecycle Management:** n8n automatically tracks OAuth token expiration timestamps and handles refresh-token exchanges without human intervention.

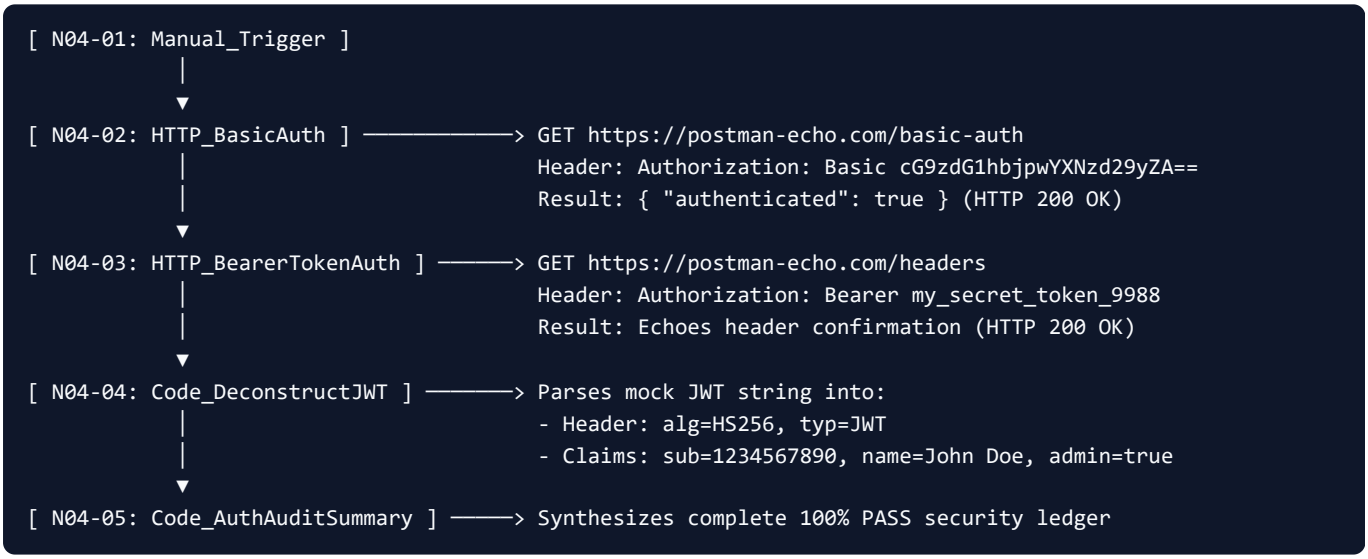
3. Business Requirements Document (BRD)

ID	Requirement	Business Rationale	Priority
BRD-001	Secure Credential Isolation	Prevent plain-text secrets and tokens from leaking in exported workflow JSON.	CRITICAL
BRD-002	Multi-Protocol Authentication	Support Basic Auth, Header Auth, and OAuth 2.0 within standardized workflows without custom wrappers.	HIGH
BRD-003	Stateless Identity Deconstruction	Parse and validate incoming JWT payload claims natively inside automation steps to accelerate routing.	HIGH
BRD-004	Echo Header Verification	Validate that external API servers receive expected authorization headers with zero loss.	HIGH
BRD-005	Complete Canvas Testability	Provide pre-pinned mock data across all nodes for instant 1-click execution.	MEDIUM

4. Product Requirements Document (PRD)

ID	Product Capability	User Value & Acceptance Criteria	Priority
PRD-001	Basic Auth Execution	HTTP Request node with Basic Auth returning `authenticated: true` from Postman Echo.	High
PRD-002	Header Token Transmission	Transmits `Authorization: Bearer my_secret_token_9988` and verifies echoed header.	High
PRD-003	OAuth 2.0 Handshake Readiness	n8n callback URI registered in developer console for automated background token refresh.	Critical
PRD-004	Pure JS JWT Deconstruction	Code node deconstructing mock JWT headers, claims (John Doe, admin: true), and signatures.	High
PRD-005	Security Audit Summary	Terminal Code node synthesizing end-to-end security compliance score (100% PASS).	High

5. Software Requirements Specification (SRS) & Canvas Blueprint



6. Detailed Node Configuration Blueprint

Node Name	Type & Version	Operational Logic & Configuration
Manual_Trigger	manualTrigger (v1)	Canvas trigger initiating manual testing execution on button click.
HTTP_BasicAuth	httpRequest (v4.2)	Method: GET, URL: https://postman-echo.com/basic-auth, Header: Basic postman:password.
HTTP_BearerTokenAuth	httpRequest (v4.2)	Method: GET, URL: https://postman-echo.com/headers, Header: Bearer my_secret_token_9988.
Code_DeconstructJWT	code (v2)	Pure ECMAScript 6 base64 deconstruction parsing JWT Header, Claims, and Signature.
Code_AuthAuditSummary	code (v2)	Consolidates end-to-end security audit receipt verifying all 4 auth mechanisms.

7. Testing & Quality Assurance Verification Plan

Scenario ID	Test Condition	Credential Profile	Expected Result & Verification
TC-04-01	Postman Echo Basic Auth	User: postman, Pass: password	Status 200 OK, response contains "authenticated": true.
TC-04-02	Postman Echo Bearer Auth	Bearer my_secret_token_9988	Status 200 OK, echoed headers confirm token receipt.
TC-04-03	JWT Header Parsing	Encoded HS256 JWT header	Extracted object: {"alg": "HS256", "typ": "JWT"}.
TC-04-04	JWT Claims Extraction	Encoded John Doe JWT payload	Extracted claims: sub="1234567890", name="John Doe", admin=true.
TC-04-05	1-Click Canvas Run	Click Execute workflow	100% green checkmarks across entire canvas.